

Summary

String Implementation and Memory Management in C Programming

This document presents a structured technical summary of Lecture 36 from the C Programming series. The lecture focuses on the internal representation of strings in C, memory allocation behavior, and the advantages and limitations of different string implementation strategies.

The core concept emphasized is that a string in C is fundamentally treated as the address of its first character. Effective string handling depends on understanding the difference between character arrays and character pointers, especially in terms of memory mutability and storage location. String literals are stored in a Read-Only Area, enabling memory optimization but restricting character-level modification. When managing multiple strings, developers must choose between 2D character arrays and arrays of character pointers, balancing memory contiguity against storage efficiency.

2. Fundamental Principles of Strings in C

2.1 String as an Address

- A string literal represents the **memory address of its first character**
- Example:
"HELLO" represents the address of 'H'

2.2 Behavior of %s in Output

- %s is commonly used with printf to print strings
- Technically, if an **address is passed to printf**, it prints characters sequentially until a **null character (\0)** is encountered

2.3 Pointer Arithmetic on Strings

Since strings are addresses, arithmetic operations are valid:

- "HELLO" + 1 → Address of 'E' → Output: ELL0

- "HELLO" + 2 → Address of 'L' → Output: LL0

3. String Implementation Strategies

Strings in C can be implemented using character arrays or character pointers, each with distinct memory behaviors.

3.1 Comparison of Arrays and Pointers

Feature	Character Array (<code>char arr[]</code>)	Pointer to Character (<code>char *ptr</code>)
Address Nature	Array name is a constant address	Pointer is a variable
String Assignment	Cannot reassign after initialization	Can be reassigned anytime
Content Modification	Individual characters can be modified	Characters cannot be modified
Memory Location	Stack or Data Segment	Read-Only Area (for literals)

4. Memory Behavior and Read-Only Area

4.1 Read-Only Memory Characteristics

String literals are stored in a **Read-Only Area**, resulting in two key properties:

1. **Immutability**
 - Modifying individual characters via a `char *` leads to error or undefined behavior
2. **Memory Optimization**
 - Identical string literals may share the same memory address
 - Multiple pointers referencing the same literal may point to the same location

4.2 Pointer Arithmetic Using ASCII Values

Character constants are treated as ASCII values, enabling arithmetic operations.

Formula Used:

`ptr + ('E' - 'A')`

- `'E' - 'A' = 4`
- Result: `ptr + 4`

Example:

If `ptr` points to `"G2025"`:

- `ptr + 4` points to `'5'`

5. Logical Comparisons in Strings

5.1 Character-Level Comparison

- Comparing individual indices evaluates ASCII values

Example:

```
str1[1] == str2[4]
```

- Returns true if both characters are identical

5.2 Array Comparison

Comparing two character arrays:

```
arr1 == arr2
```

- Always evaluates to **false**, even if contents are identical
- Reason: Arrays occupy different memory locations

5.3 Pointer Comparison

Comparing pointers pointing to the same string literal:

```
ptr1 == ptr2
```

- Often evaluates to **true**
- Reason: Compiler stores only one copy of identical literals in Read-Only memory

6. Managing Multiple Strings

6.1 Two-Dimensional Character Arrays

Structure:

```
char names[number_of_strings][max_length];
```

Characteristics:

- `names[0]` gives the address of the first string
- All strings have equal maximum length

Limitation (Internal Fragmentation):

- Short strings waste memory if one string is very long
- Example:
 - `"Balamukundan"` vs `"Ramu"`

6.2 Array of Character Pointers

Structure:

```
char *ptr[3];
```

Advantages:

- Each pointer can reference strings of different lengths
- Eliminates memory wastage seen in 2D arrays

Limitations:

- Individual characters are immutable
- Pointers can be reassigned to new strings